Software Engineering Lab 5 Gustave Perdigeon DEVOPS-13 Metrics

1. Objectif et contexte

Ce cinquième lab poursuit le projet Node.js et Express du lab précédent. L'objectif est de pratiquer JavaScript, d'implémenter une route REST correspondant à une tâche Jira, de la tester et de l'intégrer au dépôt avec le workflow GitHub Actions.

Ma contribution porte sur `GET /api/metrics`. Cette route fournit des informations sur le processus serveur et le nombre de patients actuellement enregistrés. Le travail a été réalisé dans VS Code sous Windows, sur la branche `feature/DEVOPS-13-metrics`.

- Dépôt du projet
- Tâche Jira DEVOPS-13 — Metrics
- Pull Request #9, fusionnée dans main

2. Pratique de JavaScript

Les exercices ont été exécutés avec `node js-practice-gustave.js`. Ils couvrent les variables, les chaînes avec interpolation, les conditions, les tableaux, les boucles, les objets imbriqués, la recherche et l'ajout de données, les fonctions et la gestion des erreurs avec `try/catch`. Le filtrage des patients a été réalisé avec une boucle, sans utiliser `filter()`.

La modularisation a été vérifiée avec les fichiers existants `hospitalData.js` et `main.js` : le premier exporte un tableau et une fonction, le second les importe et ajoute un médecin. L'exécution de `node main.js` a affiché le tableau mis à jour.

L'exercice asynchrone simule une récupération de données avec une Promise et `setTimeout`. Une fonction `async` utilise `await` pour attendre le résultat avant de l'afficher.

3. Implémentation de GET /api/metrics

La route est définie dans `src/routes/auto/metrics.route.js`. Elle renvoie un code HTTP **200 OK** et un objet JSON contenant trois champs.

Champ	Calcul et signification
`uptimeSeconds`	`process.uptime()` : durée écoulée depuis le démarrage du processus Node.js, en secondes.

Champ	Calcul et signification
`memoryUsedMB`	Mémoire résidente du processus, obtenue avec `process.memoryUsage().rss`, divisée par 1024^2 et arrondie à deux décimales. Malgré le nom du champ, l'unité calculée est le MiB.
`totalPatients`	Taille du tableau de patients utilisé par la route Patients.

Pour partager les données, la déclaration du tableau dans `patients.route.js` a été rendue exportable avec `export const patients`. La route Metrics importe ce même tableau. Le compteur reflète donc les ajouts effectués via `POST /api/patients`, sans maintenir une copie séparée des données.

Le chargeur existant dans `src/app.js` enregistre automatiquement les fichiers `*.route.js` du dossier `src/routes/auto`. Aucune modification de ce chargeur n'a été nécessaire pour ma route.

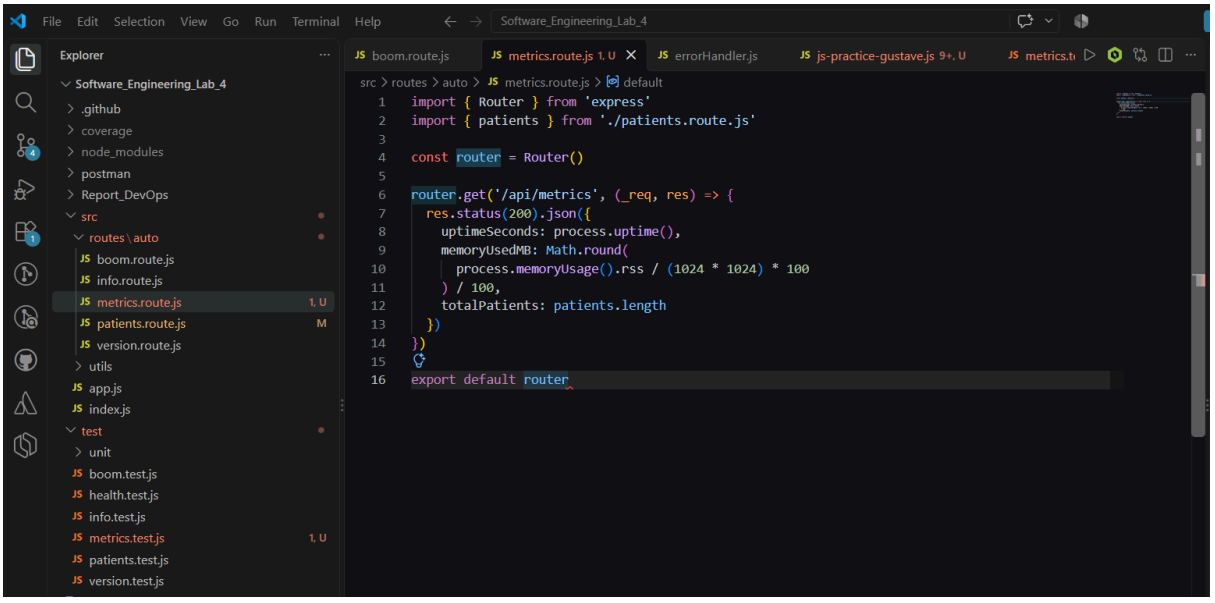


Figure 1 — Implémentation de la route Metrics dans VS Code.

4. Tests automatisés et qualité du code

Trois tests d'intégration ont été ajoutés dans `test/metrics.test.js`, avec Vitest et Supertest.

1. Vérifier le statut 200, le format JSON, une durée de fonctionnement non négative et une mémoire utilisée positive.
2. Vérifier que `totalPatients` correspond au nombre de patients renvoyés par `GET /api/patients`.
3. Ajouter un patient via POST, vérifier le statut 201, puis contrôler que le compteur augmente de 1.

La suite locale complète a obtenu **28 tests réussis dans 9 fichiers**, dont les trois nouveaux tests Metrics. Les résultats de couverture de cette exécution sont les suivants

; ils concernent la branche locale testée avant fusion.

Périmètre	Instructions	Branches	Fonctions	Lignes
Ensemble du code couvert	94,02 %	76,92 %	85,71 %	94,91 %
`metrics.route.js`	100 %	100 %	100 %	100 %

Ces résultats dépassent les seuils configurés dans le projet : 80 % pour les instructions, fonctions et lignes, et 70 % pour les branches. Les erreurs de formatage initialement signalées par ESLint ont été corrigées ; la vérification finale s'est terminée sans erreur.

Commandes de validation :

```
npm.cmd run lint
npm.cmd test -- --coverage
```

```
RUN  v4.1.11 C:/Users/gusta/Documents/Software engineering/lab4/Software_Engineering_Lab_4
     Coverage enabled with v8

 ✓ test/unit/appInfo.test.js (3 tests) 15ms
 ✓ test/unit/errorHandler.test.js (2 tests) 9ms
 ✓ test/unit/status.test.js (4 tests) 9ms
 ✓ test/boom.test.js (1 test) 44ms
 ✓ test/info.test.js (2 tests) 62ms
 ✓ test/version.test.js (2 tests) 52ms
 ✓ test/health.test.js (1 test) 61ms
 ✓ test/metrics.test.js (3 tests) 110ms
 ✓ test/patients.test.js (10 tests) 154ms

 Test Files  9 passed (9)
      Tests  28 passed (28)
   Start at  17:47:01
   Duration  1.25s (transform 480ms, setup 0ms, import 3.39s, tests 516ms, environment 2ms)

 % Coverage report from v8
-------------------|---------|----------|---------|---------|-------------------
File               | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------|---------|----------|---------|---------|-------------------
All files          |   94.02 |    76.92 |   85.71 |   94.91 |
 src               |   81.81 |     37.5 |      50 |   83.33 |
  app.js           |   94.73 |       50 |   66.66 |     100 | 29-35
  index.js         |       0 |        0 |       0 |       0 | 8-10
 src/routes/auto   |     100 |     87.5 |     100 |     100 |
  boom.route.js    |     100 |      100 |     100 |     100 |
  info.route.js    |     100 |      100 |     100 |     100 |
  metrics.route.js |     100 |      100 |     100 |     100 |
  patients.route.js|     100 |     87.5 |     100 |     100 | 18
  version.route.js |     100 |      100 |     100 |     100 |
 src/utils         |     100 |      100 |     100 |     100 |
  appInfo.js       |     100 |      100 |     100 |     100 |
  errorHandler.js  |     100 |      100 |     100 |     100 |
  status.js        |     100 |      100 |     100 |     100 |
-------------------|---------|----------|---------|---------|-------------------
```

Figure 2 — 28 tests réussis et tableau de couverture local ; la route Metrics est couverte à 100 %.

```
PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4> npm.cmd run lint

> devops-lab-student-project@1.0.0 lint
> eslint .

PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4>
```

Figure 3 — Exécution d'ESLint terminée sans erreur.

5. Validation avec Postman

Le serveur a été lancé avec `npm.cmd run dev` sur `http://localhost:3000`.

Lecture initiale et assertions

Une requête GET sur `/api/metrics` a renvoyé **200 OK**, avec les trois statistiques attendues et un compteur initial de **2 patients**. Les valeurs d'uptime et de mémoire dépendent du moment de la requête.

Trois assertions Postman vérifient le code 200, les statistiques système et la validité du compteur de patients. Les trois assertions ont réussi.

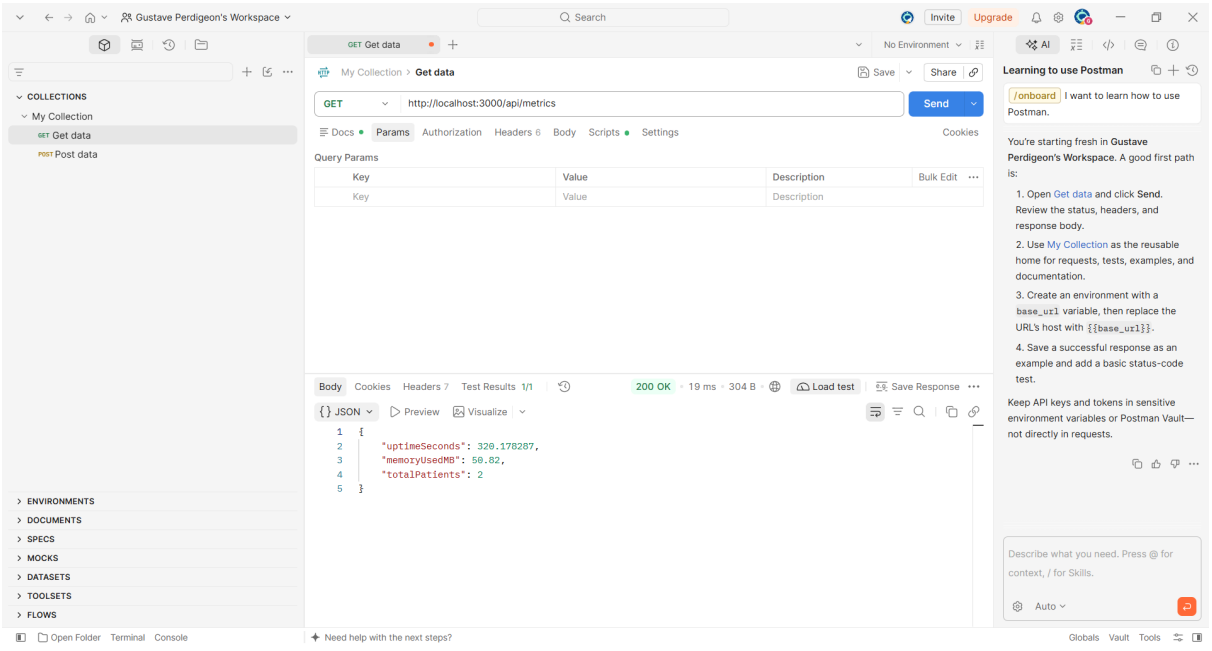


Figure 4 — Réponse initiale de GET /api/metrics : 200 OK et totalPatients égal à 2.

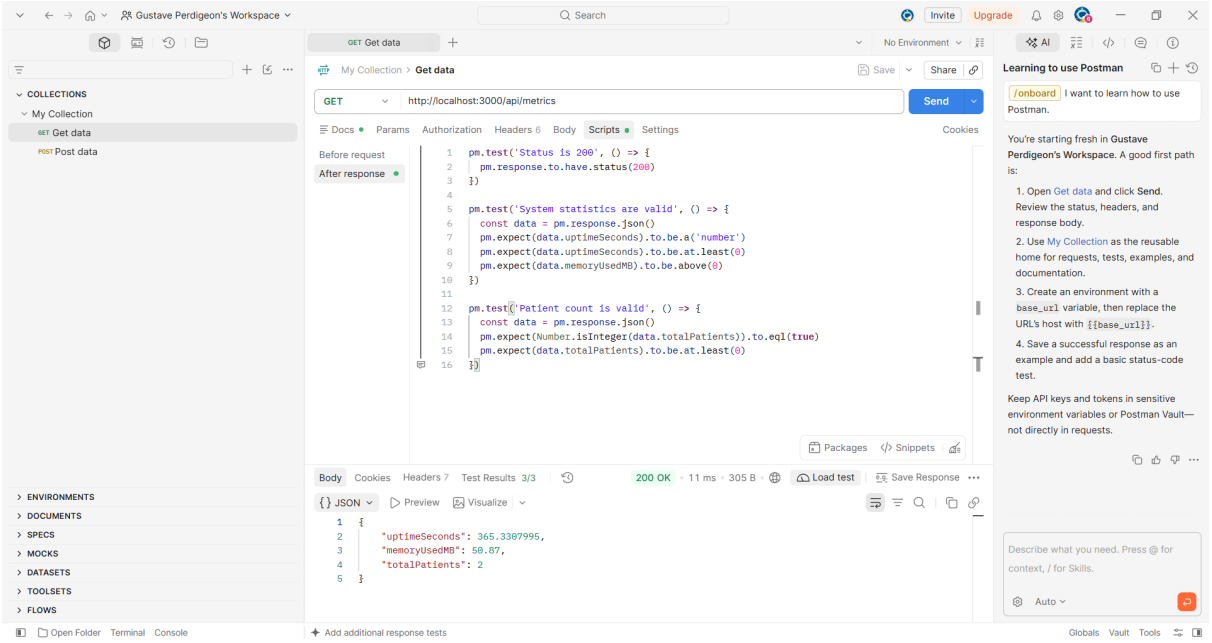


Figure 5 — Script de validation Postman et indicateur de réussite 3/3.

Vérification de la mise à jour du compteur

La requête suivante a été envoyée à `POST /api/patients` :

```
{
  "name": "Patient Metrics",
  "age": 30
}
```

L'API a répondu **201 Created**, avec le patient créé et l'identifiant **3**. Sans redémarrage du serveur, une nouvelle requête GET Metrics a ensuite renvoyé `totalPatients: 3`. Le compteur est donc passé de **2 à 3**, et les trois assertions Metrics sont restées valides.

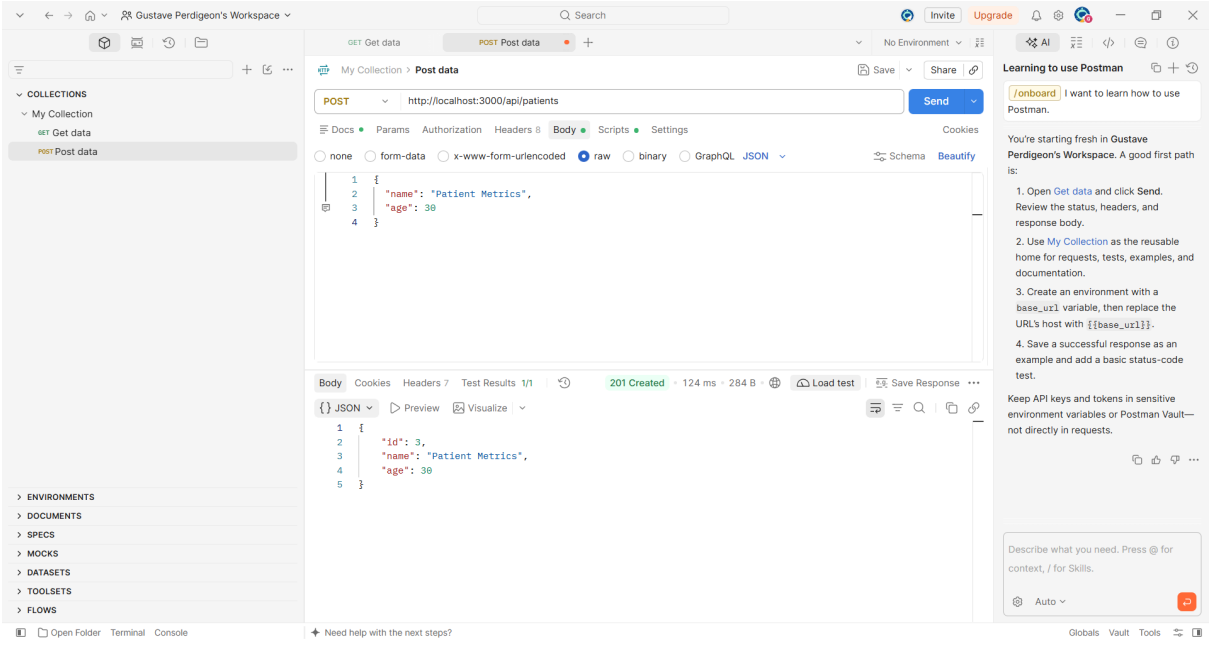


Figure 6 — Création du patient avec POST /api/patients : réponse 201 et identifiant 3.

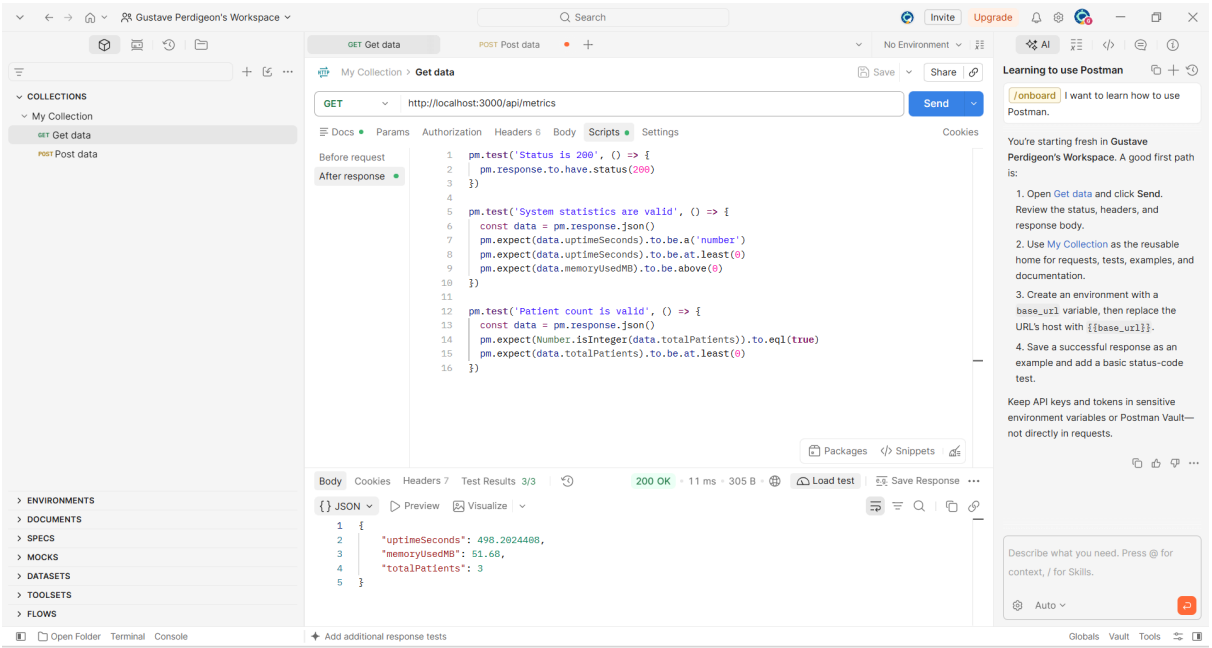


Figure 7 — Après l'ajout, GET /api/metrics renvoie totalPatients égal à 3, avec 3/3 assertions réussies.

Exécution et export de la collection

La collection **Metrics DEVOPS-13** contient les requêtes GET Metrics et POST Patient. Une exécution du Collection Runner avec une itération a obtenu **4 assertions réussies, 0 échec et 0 erreur** : trois assertions sur Metrics et une sur la création du patient.

La vérification du passage de 2 à 3 est documentée par les requêtes manuelles ci-dessus et par le test d'intégration ; le Runner exécute ici GET puis POST et ne vérifie pas lui-même cette augmentation.

La collection a été exportée au format **Postman v2.1** et ajoutée au dépôt sous `postman/Metrics_DEVOPS-13.postman_collection.json`.

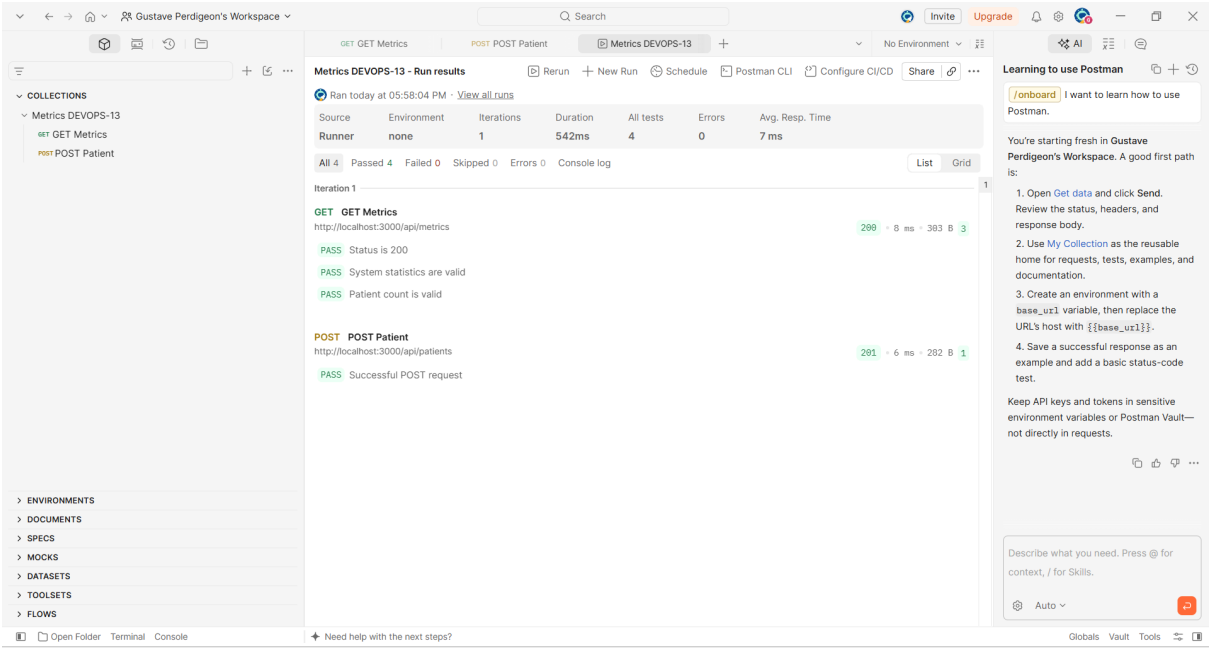


Figure 8 — Bilan du Collection Runner : quatre assertions réussies et aucune erreur.

6. Workflow Git, GitHub Actions et Jira

Les cinq fichiers livrés sont :

- `js-practice-gustave.js` : exercices JavaScript individuels.
- `src/routes/auto/metrics.route.js` : nouvelle route REST.
- `src/routes/auto/patients.route.js` : export du tableau partagé.
- `test/metrics.test.js` : trois tests d'intégration.
- `postman/Metrics_DEVOPS-13.postman_collection.json` : collection exportée.

Le travail a été enregistré dans le commit 51cf66c, intitulé `feat(DEVOPS-13): add metrics endpoint, tests and Postman collection`, puis poussé sur la branche dédiée.

La **Pull Request #9** cible `main`. Le check **CI / build-and-test (pull_request)** a réussi en **21 secondes** et GitHub n'a signalé aucun conflit. La PR a ensuite été fusionnée dans `main`. La réussite CI est une vérification distincte de la suite locale de 28 tests présentée plus haut.

La tâche Jira **DEVOPS-13 — Metrics** est liée à la PR et affiche la fusion ainsi qu'un build réussi. La capture conserve la traçabilité entre la tâche, la branche, les commits et la livraison.

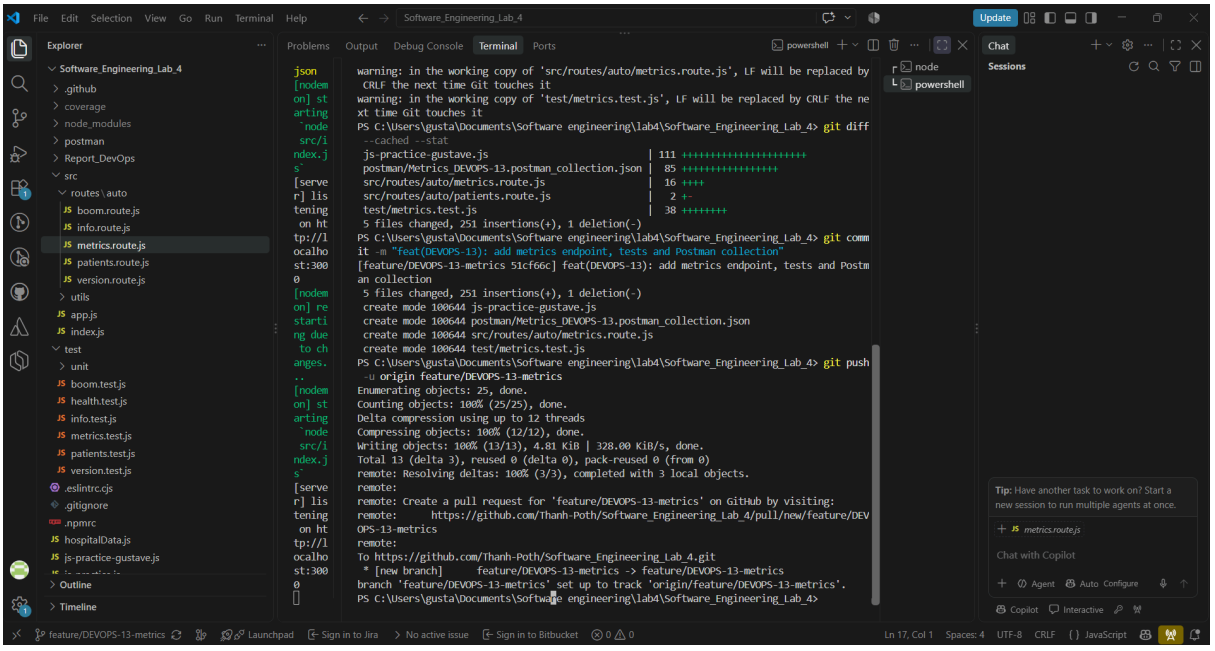


Figure 9 — Commit des cinq fichiers et push réussi de la branche feature/DEVOPS-13-metrics.

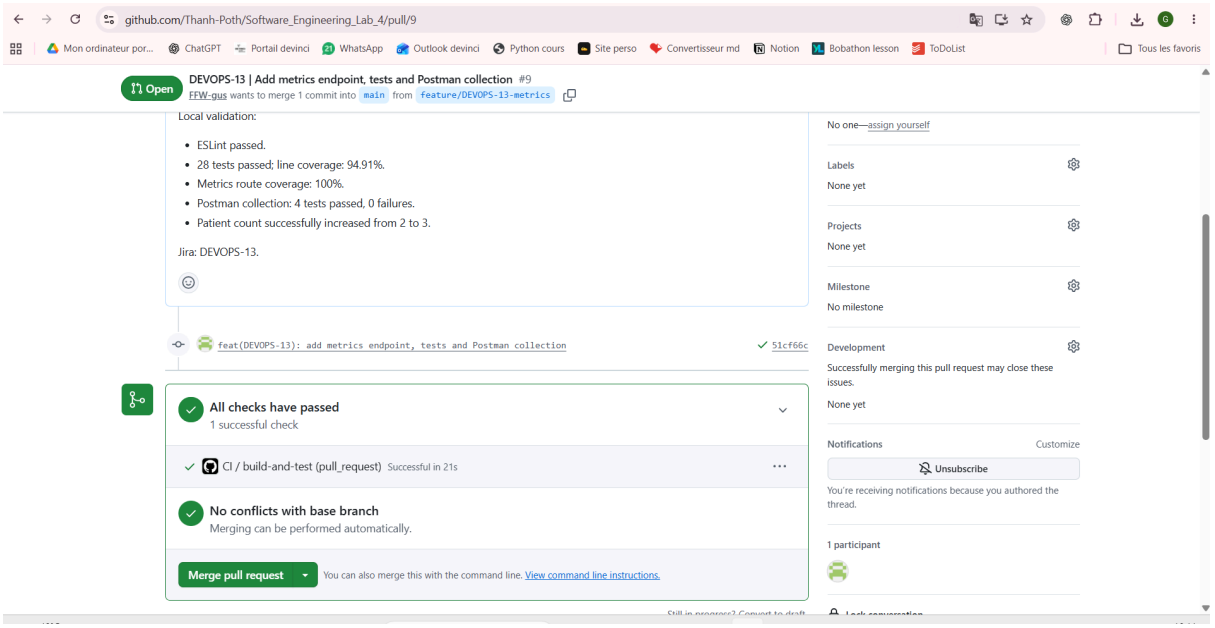


Figure 10 — Pull Request #9 : check CI réussi et absence de conflit avec main.

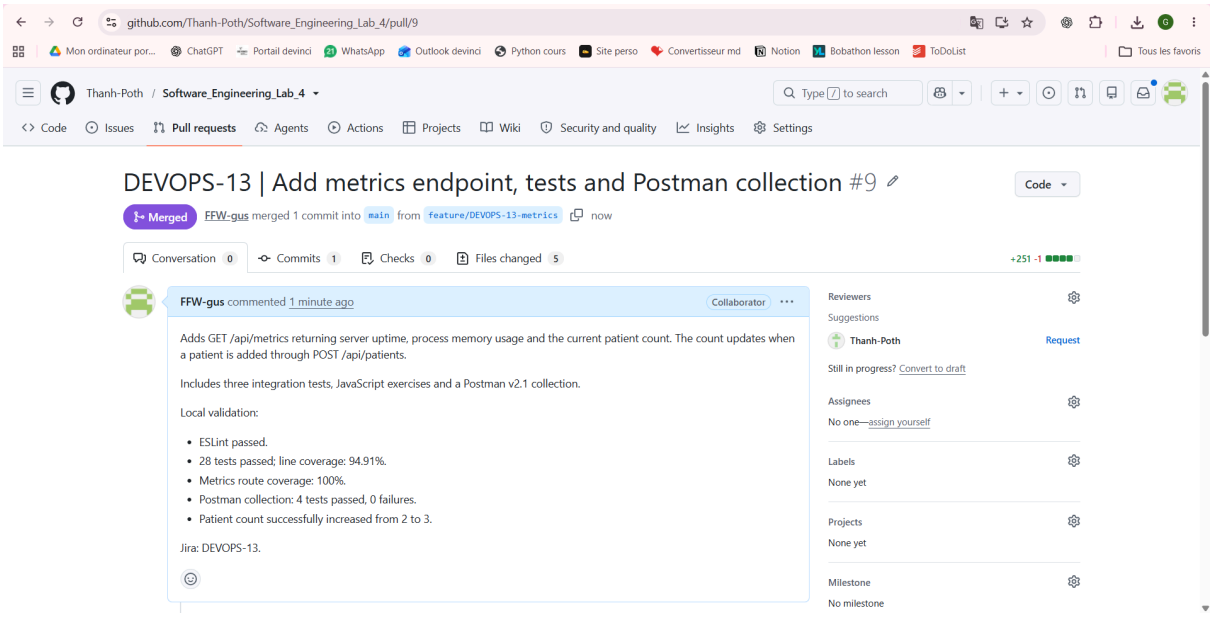


Figure 11 — Pull Request #9 fusionnée dans main.

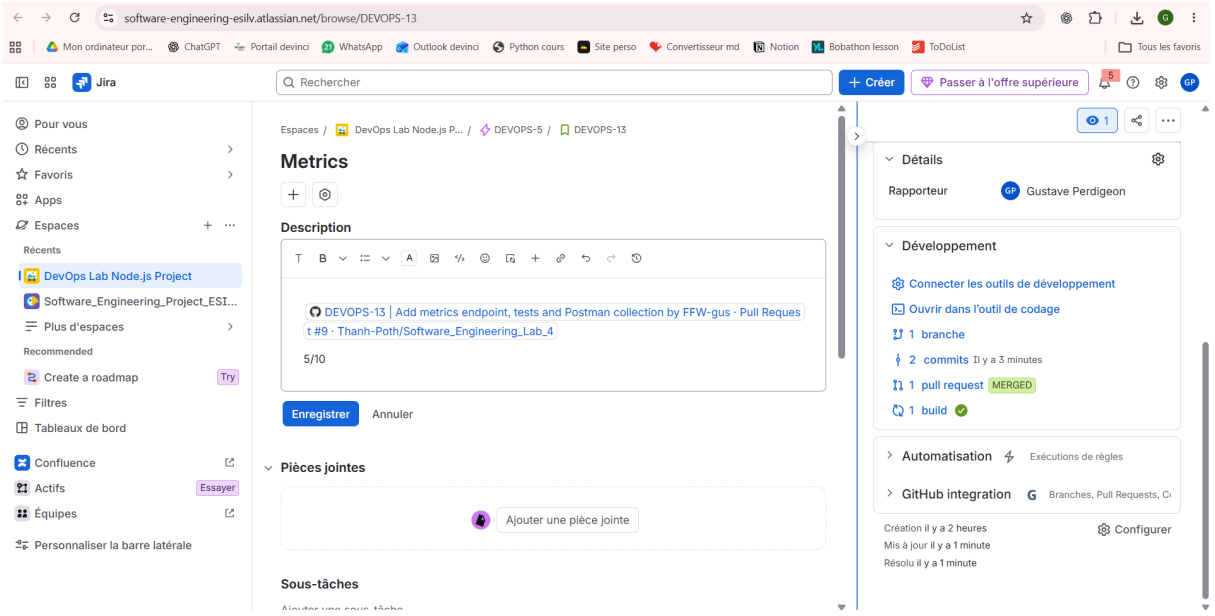


Figure 12 — Tâche Jira DEVOPS-13 liée à la PR fusionnée et au build réussi.

7. Bilan et limites

La fonctionnalité Metrics fournit une réponse JSON accessible par HTTP, expose les mesures du processus Node.js et reflète les ajouts de patients. Elle a été validée par les tests d'intégration, les vérifications Postman et le pipeline CI, puis intégrée à la branche principale.

Les données patients restent stockées en mémoire : elles sont réinitialisées au redémarrage et le compteur concerne uniquement cette instance du serveur. L'uptime et la mémoire décrivent le processus Node.js, et non l'ensemble de la machine. Aucune

persistance en base de données ni interface graphique n’a été ajoutée dans cette contribution.